Sesion 7: Cierre Integrador

Metricas DORA · Estrategia QA Cloud-Native · Mapa del modulo

Certificacion 5: QA Cloud-Native - Automatizacion Inteligente con IA Generativa

1. De donde venimos
2. Mapa integrador (Dias 1-6)
3. Metricas DORA
4. Cobertura y MTTR
5. Estrategia: la piramide
6. Donde encaja cada herramienta
7. Que tener en cuenta al medir
8. Ejercicio integrador
9. Tablero en Grafana
10. Cierre del modulo

1. De donde venimos

Seis dias, una sola app, un ciclo de vida completo de QA cloud-native

La historia en una frase

Partimos de UNA tienda web con API y la acompanamos por todo su ciclo: la gestionamos en Azure DevOps, la automatizamos con Playwright + Screenplay + BDD apoyados por IA (MCP), la ejecutamos en la nube (Sauce Labs), la contenerizamos y validamos sus contratos (Pact), y hoy la **consolidamos**: medimos que tan bien entregamos y definimos la estrategia.

Automatizar no es el fin — es el medio

Tener pruebas automatizadas, contenedores y pipelines no sirve de nada si no sabemos **si estamos entregando mejor**. El objetivo real de todo el modulo es **entregar software de calidad, rapido y de forma sostenible**. Para saber si lo estamos logrando, un equipo de QA debe poder responder tres preguntas:

- **¿Vamos bien?** — se responde con metricas (DORA, cobertura, MTTR).
- **¿Estamos probando donde toca?** — se ordena con la piramide de testing.
- **¿Como encaja cada herramienta?** — se mapea sobre la app de la tienda.

Esas tres preguntas son el hilo de hoy: pasamos de "construir pruebas" a **medir y decidir** con base en datos.

2. Mapa integrador: todo gira alrededor de la Tienda

Nada fue un tema suelto; cada dia agrego una capa al mismo producto

Dia	Capa que agrego	Herramienta principal	Que corre de verdad
1	Datos de prueba deterministas	Faker (semilla fija)	Si
2	API + pipeline + quality gate	Node + AWS + SECURITY_GATE	Si
3	Gestion de casos y CI/CD	Azure DevOps (Test Plans, Pipelines)	Pipeline local-first + demo
4	E2E automatizado + IA	Playwright + Screenplay + BDD + MCP	Si
5	Ejecucion en la nube	Pipeline + Sauce Labs	Pipeline si + Sauce demo

Dia	Capa que agrego	Herramienta principal	Que corre de verdad
6	Contenedores + contract testing + self-healing	Podman + Pact + Serenity/Healenium	Si
7	Medicion y estrategia	DORA + piramide	Hoy

> 💡 La frase que resume el modulo: **"una app, siete capas, un mismo ciclo de vida"**. Si un participante solo recuerda una cosa, que sea esta: las herramientas no compiten, se complementan en etapas distintas del ciclo.

PARTE A

¿Vamos bien? — Medir con metricas

No se puede mejorar lo que no se mide. Las metricas DORA son el estandar de la industria para medir el rendimiento de entrega de software, respaldadas por la investigacion de Google (programa DORA / informe State of DevOps).

Secciones 3-4 · Metricas que un lider de QA debe saber leer

3. Las 4 metricas DORA

Dos miden velocidad, dos miden estabilidad. Juntas evitan optimizar una a costa de la otra.

Que es DORA

DORA (DevOps Research and Assessment) es un programa de investigacion de Google que identifico 4 metricas que predicen el rendimiento de un equipo de entrega de software. La idea clave: **velocidad y estabilidad no se oponen**; los equipos de elite son rapidos Y estables a la vez.

1. Frecuencia de despliegue

```
deploys / periodo
```

Velocidad. Cada cuanto llevas cambios a produccion. Elite: varias veces al dia. Bajo: menos de una vez al mes.

2. Lead time for changes

```
commit → produccion (tiempo)
```

Velocidad. Cuanto tarda un cambio desde que se escribe hasta que llega a produccion. Elite: menos de 1 dia.

3. Change failure rate

```
deploys_fallidos / deploys_totales
```

Estabilidad. Que porcentaje de despliegues causa un fallo en produccion (rollback, hotfix). Elite: 0-15%.

4. MTTR

```
tiempo medio de recuperacion
```

Estabilidad. Cuanto tardas en recuperarte de un fallo en produccion. Elite: menos de 1 hora.

Como las alimenta QA (la conexion con lo que hicimos)

- **Frecuencia ↑ y Lead time ↓:** gracias a un pipeline con pruebas automatizadas (Dias 3-5) que da confianza para desplegar seguido y rapido.
- **Change failure rate ↓:** gracias a los quality gates, el contract testing (Pact, Dia 6) y el E2E que atrapan rupturas *antes* de produccion.
- **MTTR ↓:** gracias a pruebas que detectan el fallo rapido y a infraestructura reproducible (contenedores, Dia 6) que permite recrear y revertir en minutos.

Niveles de rendimiento (referencia DORA)

Metrica	Elite	Alto	Medio	Bajo
Frecuencia de deploy	varias/dia	1/dia–1/sem	1/sem–1/mes	< 1/mes
Lead time	< 1 dia	1 dia–1 sem	1 sem–1 mes	> 1 mes
Change failure rate	0–15%	16–30%	16–30%	> 30%
MTTR	< 1 hora	< 1 dia	1 dia–1 sem	> 1 sem

No memorices los numeros: entiende la idea. Lo importante es la **tendencia** de tu equipo en el tiempo, no compararte con una tabla.

4. Cobertura, flaky y el peligro de las metricas vanidosas

Metricas de QA que complementan DORA — con una advertencia

Cobertura de codigo

% de lineas/ramas ejercitadas por las pruebas. Util como **senal**, peligrosa como **meta**: 100% de cobertura no significa 0 bugs. Mide lo que se ejecuta, no lo que se valida bien.

Cobertura de requisitos

% de casos de negocio cubiertos por pruebas (trazabilidad caso → test). Mas valiosa que la cobertura de codigo para QA: responde "¿probamos lo que importa al negocio?".

Tasa de flaky tests

% de pruebas que fallan de forma intermitente sin cambio de codigo. Alta = pierdes confianza en la suite. El self-healing (Dia 6) ataca una causa comun: selectores fragiles.

MTTR de la suite

Cuanto tardas en arreglar una prueba rota. Si una prueba rota tarda dias en arreglarse, el equipo empieza a ignorarla. Pruebas mantenibles = MTTR bajo.

⚠️ **Metricas vanidosas:** "numero total de pruebas" o "100% de cobertura" se ven bien en un reporte pero no dicen nada de la calidad. Una suite de 5000 pruebas lentas y flaky es peor que 200 pruebas rapidas y fiables. Mide **valor**, no volumen.

PARTE B

¿Probamos donde toca? — La estrategia

Tener muchas herramientas no es una estrategia. La estrategia es saber **que probar en cada nivel** y **donde encaja cada herramienta** que vimos en el modulo.

Secciones 5-6 · La piramide de testing aplicada a la Tienda

5. La piramide de testing cloud-native

Muchas pruebas rapidas abajo, pocas pruebas lentas arriba

E2E / UI
pocas · lentas · caras · fragiles

Contract testing (Pact)

valida el acuerdo entre servicios

Integracion / API
mas rapidas, prueban varios componentes

Unitarias
muchas · rapidas · baratas · estables

Como leer la piramide

- **Abajo (base ancha):** muchas pruebas unitarias. Son rapidas, baratas y estables. La mayoria de tus pruebas viven aqui.
- **Medio:** pruebas de integracion/API y **contract testing**. El contract testing (Pact, Dia 6) es la pieza cloud-native: valida el acuerdo entre servicios sin levantar todo el sistema.
- **Arriba (punta estrecha):** pocas pruebas E2E/UI. Son valiosas (prueban el flujo real del usuario) pero lentas, caras y fragiles. Por eso **pocas y bien elegidas**.

⚠️ **El antipatron del "cono de helado":** la piramide invertida — muchas pruebas E2E y pocas unitarias. Es lento, flaky y caro de mantener. Si tu suite tarda horas y falla sin razon, probablemente tienes un cono de helado.

💡 El self-healing (Healenium, Dia 6) NO justifica tener mas E2E. Reduce el mantenimiento de las E2E que *si* necesitas, pero la estrategia sigue siendo: pocas E2E, bien elegidas.

6. Donde encaja cada herramienta del modulo

El mapa completo: cada herramienta en su nivel y su momento

Nivel / Etapa	Herramienta	Que resuelve
Datos de prueba	Faker (Dia 1)	Datos deterministas para pruebas repetibles
Unitario / API	pytest, tests de API (Dia 2)	Logica interna y endpoints de la Tienda
Contract testing	Pact (Dia 6)	Acuerdo Tienda ↔ Notificaciones sin E2E pesado
E2E / UI	Playwright+Screenplay (Dia 4) · Serenity+Healenium (Dia 6)	Flujo real del usuario; self-healing baja el mantenimiento
Gestion de casos	Azure Test Plans (Dia 3)	Organizar y trazar casos de prueba
CI/CD	Azure Pipelines / GitHub Actions (Dias 3,5)	Ejecutar todo lo anterior en cada cambio
Ejecucion en la nube	Sauce Labs (Dia 5)	Navegadores y dispositivos reales a escala
IA / mantenimiento	MCP (Dia 4)	Agente que lee, ejecuta y ayuda a reparar pruebas
Empaquetado	Podman / contenedores (Dia 6)	Entorno reproducible: corre igual en PC, CI y nube
Medicion	DORA + cobertura (Dia 7)	Saber si todo lo anterior esta funcionando

💡 Esta tabla es el entregable mental del modulo. Un QA cloud-native no es "el que sabe Playwright": es quien sabe **que herramienta usar en cada nivel y por que**.

7. Que tener en cuenta al medir

Buenas practicas y trampas comunes al trabajar con metricas

Mide tendencias, no numeros aislados

Una metrica suelta no dice nada; lo que importa es **como evoluciona en el tiempo**. ¿El lead time baja mes a mes? ¿El change failure rate sube tras un cambio de proceso? La direccion vale mas que el valor puntual.

Las 4 DORA van juntas

Nunca optimices una a costa de otra. Desplegar mas rapido (frecuencia ↑) pero rompiendo mas (change failure rate ↑) NO es mejorar. Lee velocidad y estabilidad **como un conjunto**.

Cuidado con las metricas vanidosas

"Numero total de pruebas" o "100% de cobertura" se ven bien pero no reflejan calidad. Prefiere metricas que midan **valor real**: cobertura de requisitos, flaky rate, tiempo de feedback.

No uses metricas para castigar

Si el equipo siente que las metricas sirven para senalar culpables, las empezara a manipular. Son para **aprender y mejorar el sistema**, no para evaluar personas.

Automatiza la recoleccion

Si medir es manual, nadie lo hace. Los datos deben salir solos del pipeline (CI/CD), de Git y del sistema de incidentes. Metrica que requiere trabajo manual = metrica que se abandona.

Conecta la metrica con una accion

Cada metrica debe responder "¿y si esta mal, que hago?". Lead time alto → revisa el pipeline. Change failure rate alto → refuerza quality gates y contract testing. Metrica sin accion = ruido.

8. Ejercicio integrador — Calculadora DORA

Medir de verdad: un script que calcula las 4 metricas, en contenedor

La idea

Para que las metricas DORA no se queden en teoria, las **calculamos**. Un script Node lee un historial de despliegues (`deployments.json`) y calcula las 4 metricas, clasificando cada una en Elite/Alto/Medio/Bajo. Corre en un contenedor Podman, como todo el modulo.

🪛 EJERCICIO PRACTICO

Ejercicio — Calcular DORA sobre la app Tienda

📂 Carpeta: `dia7/proyecto/` · archivo de datos: `deployments.json`

1. **Construye y corre la calculadora (datos de ejemplo):**
   `podman build -t dora .`
   `podman run --rm dora`
   ✓ *Verifica (resultado real verificado):* las 4 metricas en **Elite** — 2.05 deploys/dia, lead time 2.0 h, change failure rate 10% (1/10), MTTR 0.7 h.

2. **Rompe la estabilidad:** en `deployments.json` , marca 4 de los 10 deploys con `"failed": true` (y agrega su `restored_at`). Reconstruye y corre de nuevo.
   ✓ *Verifica:* el **change failure rate** sube a 40% y cae a nivel **Bajo**. Las dos metricas de velocidad no cambian: velocidad y estabilidad son ejes independientes.

3. **Rompe la velocidad:** en un deploy, pon el `committed_at` dos semanas antes del `deployed_at` . Corre de nuevo → el **lead time** se dispara.

4. **Conecta con el modulo:** para cada metrica que empeoro, nombra que practica de los Dias 1-6 la mejoraria (pipeline + quality gates, contract testing, contenedores reproducibles, self-healing...).

Usar tus propios datos sin reconstruir: `podman run --rm -v "${PWD}/deployments.json:/data/deployments.json:ro" -e DEPLOYMENTS_FILE=/data/deployments.json dora`

De donde saldrian estos datos en la vida real

- `deployed_at` → del CI/CD (Azure Pipelines, GitHub Actions).
- `committed_at` → del historial de Git (`git log`).
- `failed` / `restored_at` → del sistema de incidentes o de los rollbacks del pipeline.

Aqui los damos en un JSON para enfocarnos en el **calculo y la lectura**, que es lo que un lider de QA debe saber hacer.

9. Tablero DORA en Grafana (visualizacion real)

Del número en consola al dashboard que ve todo el equipo

Por que un tablero

Un número en la terminal lo ve quien corre el comando. Un **tablero** lo ve todo el equipo, en tiempo real, con colores que gritan "esto está mal". **Grafana** es el estándar de la industria para esto, y conecta perfecto con la calculadora: ella produce los datos, Grafana los pinta.

Arquitectura (todo en contenedores Podman)

flujo de datos



- **metricas-dora** (Node): recalcula las 4 métricas en cada petición y las expone como JSON en `/metricas` .
- **grafana-dora**: con el plugin **Infinity** (lee JSON por HTTP), datasource y dashboard DORA **pre-provisionados** (se cargan solos al arrancar).

🪛 EJERCICIO PRACTICO

Levantar el tablero

📁 Carpeta: `dia7/proyecto/`

comandos podman

```
# 1) construir la imagen del servidor de metricas (una vez)
#    (Windows: podman-compose ignora 'dockerfile:', por eso se construye aparte)
podman build -t dora-metricas -f Dockerfile.metricas .

# 2) levantar servidor de metricas + Grafana
podman compose up -d

# 3) abrir el tablero
#    http://localhost:3001   (admin/admin, o entra como anonimo)
#    dashboard "DORA - app Tienda"

# ver el JSON crudo que consume Grafana (opcional):
#    http://localhost:8900/metricas

# 4) apagar
podman compose down
```

✓ *Verificado end-to-end:* el plugin Infinity se instala, el datasource y el dashboard se provisionan solos, y una query de Grafana contra el servidor de métricas devuelve los datos reales (las 4 en **Elite** con el JSON de ejemplo).

Qué verás

Cuatro paneles (uno por métrica) coloreados por nivel — verde = Elite, rojo = Bajo — más una tabla resumen con valores y clasificación.

✏️ **EJERCICIO PRACTICO**

Mover el dashboard — cambiar los valores

El JSON vive DENTRO de la imagen (en Windows el bind mount de un archivo no sirve). Ciclo: editar → reconstruir → RECREAR el contenedor → recargar Grafana.

desde dia7/proyecto/

```
# 1) edita deployments.json (ej: marca 4 deploys con "failed": true)
# 2) reconstruye la imagen con el JSON nuevo
podman build -t dora-metricas -f Dockerfile.metricas .
# 3) RECREA el contenedor (compose up REUSA el viejo: hay que borrarlo)
podman rm -f metricas-dora
podman compose up -d metricas-dora
# 4) recarga el dashboard en Grafana (F5)
```

✓ *Verificado:* con 1 deploy fallido → change failure rate 10% (Elite, verde); con 4 fallidos → 40% (Bajo, rojo). Confirmar sin abrir Grafana: `podman exec metricas-dora wget -qO- http://localhost:8900/metricas`. Así ves el ciclo **dato** → **cálculo** → **visualización**.

💡 Grafana descarga el plugin Infinity en el primer arranque (necesita internet esa vez). Usa el puerto **3001** porque el **3000** lo ocupa la app Tienda.

⚠️ **Version del plugin Infinity (importante):** debe ser compatible con Grafana. Infinity **4.x** exige Grafana **>=11.6.11**; con Grafana 11.3.0 falla con `404 react/jsx-runtime` y los paneles quedan en **"No data"**. Por eso fijamos **Infinity 3.7.1** (la ultima compatible con 11.3.0): `GF_INSTALL_PLUGINS=yesoreyeram-infinity-datasource 3.7.1`.

10. Cierre del modulo

Que te llevas de estos siete dias

La idea que resume todo

Un QA cloud-native no es quien domina una herramienta, sino quien entiende el **ciclo completo**: datos, automatizacion, IA, ejecucion en la nube, contenedores, contratos y medicion. Todo sobre una misma app, todo reproducible, todo medible. Eso es lo que construimos en estos siete dias.